

Agenci kodujący od zera

Co to jest, jak z tego korzystać i dlaczego zmienia sposób, w jaki powstaje oprogramowanie.

Zeszyt 1 z 4 · Pakiet „Jak powstała aplikacja tic-bot-toe”

Poziom: od zera — nie zakładamy żadnej wiedzy o AI ani o programowaniu.

Stan na 18.07.2026

Czego się tu nauczysz

Ten zeszyt tłumaczy pojęcia, których użyjemy w całym pakiecie. Po jego przeczytaniu:

- będziesz wiedzieć, czym różni się **agent kodujący** od zwykłego czatu z AI,
- zrozumiesz dwa narzędzia, których użyjemy: **Claude Cowork** (do myślenia) i **Claude Code** (do budowania),
- poznasz pętlę pracy: **brief** → **plan** → **realizacja** → **weryfikacja** i zobaczysz, gdzie w niej jest miejsce człowieka,
- oswoisz słownik (kontekst, tokeny, prompt, klucz API, halucynacja) na tyle, by resztę pakietu czytać bez potykania się o żargon,
- będziesz wiedzieć, jak zacząć **bez ponoszenia kosztów** albo za grosze.

Nie musisz niczego instalować, żeby przeczytać ten zeszyt. Instalacje są w Zeszycie 3.

1. Zaczniemy od obrazu, nie od definicji

Wyobraź sobie, że zatrudniasz bardzo szybkiego, odczytanego stażystę. Nie musisz mu dyktować każdej litery — mówisz, **co** ma zrobić, a on sam otwiera pliki, pisze, sprawdza, poprawia i pokazuje wynik. Możesz go poprosić: „dodaj ekran logowania i upewnij się, że działa”. On przeczyta istniejący kod, dopisze nowy, uruchomi testy i wróci z raportem.

Tym stażystą jest **agent kodujący**. To program oparty na dużym modelu językowym (LLM — o tym za chwilę), który nie tylko *odpowiada tekstem*, ale ma **narzędzia**: potrafi czytać i zapisywać pliki, uruchamiać polecenia, przeszukiwać projekt, a nawet otwierać stronę w przeglądarce, żeby sprawdzić, czy zmiana zadziałała.

To jest kluczowa różnica, więc powtórzmy ją wprost.

2. Czat kontra agent – najważniejsze rozróżnienie

WSKAZÓWKA

Czat z AI (jak zwykłe okno rozmowy) → zadajesz pytanie, dostajesz tekst w odpowiedzi. Kopiowanie kodu tam i z powrotem to Twoja robota.

Agent kodujący → dostaje cel i **sam działa** w Twoim projekcie: otwiera pliki, wprowadza zmiany, uruchamia testy, commituje. Ty go prowadzisz i sprawdzasz efekt.

	Czat z AI	Agent kodujący
Co dostajesz	tekst / kod do skopiowania	zmiany wprowadzone wprost w projekcie
Kto otwiera pliki	Ty	agent
Kto uruchamia testy	Ty	agent
Kto pamięta cały projekt	Ty przypominasz	agent czyta pliki, gdy potrzebuje
Twoja rola	wykonawca	reżyser i recenzent

Cały ten pakiet dotyczy pracy z **agentem**, konkretnie z **Claude Code**. Ale zaczniemy od jego spokojniejszego kuzyna.

3. Dwa narzędzia, dwie różne role

W projekcie tic-bot-toe (opisanym w Zeszycie 2) użyto dwóch narzędzi. Nie są konkurencją — robią co innego.

WSKAZOWKA

Claude Cwork — miejsce do **myślenia**. Rozmawiasz, ścierasza pomysły, prosisz o analizę „czy ktoś już to robi”, dochodzisz do **specyfikacji** — dokumentu opisującego, co ma powstać. Cwork pracuje na dokumentach i researchu, nie grzebie w kodzie produkcyjnym.

Claude Code — miejsce do **budowania**. Uruchamiasz go w folderze z projektem; on czyta specyfikację i pisze aplikację: kod, testy, konfigurację, commity.

Analogia: **Cwork to architekt i projektant**, z którym ustalasz, co i po co budujemy. **Claude Code to ekipa budowlana**, która stawia budynek zgodnie z projektem. Najlepsze efekty daje ich połączenie — najpierw dobry projekt, potem wykonanie.

4. Pętla pracy z agentem

Praca z agentem to nie „jeden magiczny prompt”. To powtarzalna pętla. Zapamiętaj ją — wraca w każdym zeszycie.

1. **Brief** — mówisz, co ma powstać. Im konkretniej, tym lepiej (patrz sekcja 6).
2. **Plan** — agent (albo Ty z jego pomocą) rozbija cel na etapy. Duże zadania robi się kawałkami, nie naraz.
3. **Realizacja** — agent pisze kod etap po etapie.

4. **Weryfikacja** — sprawdzasz, czy działa: testy, uruchomienie aplikacji, kliknięcie na własne oczy. Jeśli nie — wracasz do punktu 3 z poprawką.

OK - PUNKT KONTROLNY

Po każdym większym kroku powinno być „zielono” (testy przechodzą, aplikacja się uruchamia), zanim ruszysz dalej. To najważniejsza zasada higieny — buduje się warstwami, nie wszystko naraz.

5. Gdzie w tym wszystkim jest człowiek?

Skoro agent pisze kod — po co Ty? Okazuje się, że Twoja rola jest **ważniejsza**, nie mniej ważna. Agent jest szybki i pracowity, ale nie wie, **co** naprawdę chcesz osiągnąć ani **co się liczy**. To Ty:

- **podejmujesz decyzje** — jaki pomysł, jakie priorytety, jakie kompromisy,
- **piszesz brief** i specyfikację (albo prowadzisz Cowork, by je z Tobą napisał),
- **recenzujesz** — czytasz, co powstało, i mówisz „to źle, popraw tak”,
- **weryfikujesz na żywo** — klikasz w aplikację i sprawdzasz, czy naprawdę robi to, co miała.

W projekcie z Zeszytu 2 człowiek napisał **zero linii kodu** — a mimo to był autorem całości. Bo autorstwo to kierunek i decyzje, nie klepanie znaków.

6. Dlaczego „jak prosisz” decyduje o wyniku

Agent zrobi dokładnie to, o co poprosisz — więc niejasna prośba daje niejasny wynik. Porównaj:

UWAGA

Słabo: „zrób mi grę”. Agent musi zgadywać wszystko: jaką grę, dla kogo, w czym, jak ma wyglądać. Zgadnie — i pewnie nie tak, jak chciałeś.

OK - PUNKT KONTROLNY

Dobrze: „zrób grę w kółko i krzyżyk jako stronę internetową; dwóch graczy na jednym ekranie; po wygranej pokaż, kto wygrał, i przycisk »zagraj ponownie«; ma działać na telefonie”. Teraz agent wie, co budować.

Nie musisz znać terminów technicznych — musisz jasno powiedzieć, **co ma się dziać i co się liczy**. Reszty (jaką technologię wybrać) agent doradzi. W Zeszytcie 4 dostaniesz gotowe prompty do skopiowania, więc zobaczysz ten wzorec w praktyce.

7. Słownik bez żargonu

Tyle pojęć wystarczy na cały pakiet. Wracaj tu, gdy coś się pojawi.

Pojęcie	Po ludzku
LLM (model językowy)	„mózg” AI — program wytrenowany na ogromnej ilości tekstu, który przewiduje kolejne słowa. Pod spodem działają czat i agent.
Model	konkretna wersja tego mózgu, np. Opus 4.8 czy Fable 5. Różnią się szybkością, kosztem i „bystrością”.
Prompt / brief	to, co piszesz do AI. Instrukcja lub pytanie.
Kontekst	ile informacji model „widzi” naraz (Twoje polecenia + pliki, które czytał). Ograniczony — jak biurko, na którym mieści się tylko tyle kartek.
Token	kawałek tekstu (mniej więcej sylaba/krótkie słowo). Modele liczą pracę w tokenach; od nich zależy koszt.
Klucz API	hasło dostępu do modelu przez program. Traktuj jak hasło — nie pokazuj nikomu (sekcja 8).
Halucynacja	gdy model pewnym tonem podaje coś nieprawdziwego. Dlatego weryfikujesz wyniki.
Repozytorium (repo)	folder z projektem, wraz z historią zmian (patrz: git).
git / commit	system zapisujący historię zmian w kodzie. „Commit” to jeden zapisany krok, który zawsze można cofnąć.
Terminal	okno, w którym wpisuje się polecenia tekstem. Agent często z niego korzysta za Ciebie.

WSKAZÓWKA

Nie musisz tego wkuć na pamięć. Wystarczy, że wiesz, że **token = jednostka kosztu**, **kontekst = pamięć robocza modelu**, a **klucz API = hasło**. Do reszty wrócisz w razie potrzeby.

8. Bezpieczeństwo i koszty – zanim klikniesz cokolwiek

Dwie rzeczy budzą najwięcej obaw u początkujących. Rozbrójmy je.

Klucze API to hasła

Klucz API to ciąg znaków, który daje dostęp (często płatny) do modelu. Zasady są proste:

- **nigdy** nie wklejaj klucza do losowych stron ani nie wysyłaj go w wiadomości,
- trzymaj go w pliku `.env` (nauczysz się w Zeszytcie 3) — to plik, którego **nie** wgrywa się do repozytorium,
- jeśli podejrzewasz, że klucz wyciekł — skasuj go w panelu dostawcy i wygeneruj nowy.

UWAGA

W aplikacji z tego pakietu przyjęto zasadę, że klucz użytkownika żyje **wyłącznie w jego przeglądarce** i nigdy nie trafia na cudzy serwer. To dobry wzorzec — pilnuj, komu i gdzie powierzasz klucze.

Skąd się biorą koszty i jak zacząć za darmo

Płacisz zwykle za **tokeny** — im więcej model czyta i pisze, tym drożej. Ale zacząć da się **bez wydawania pieniędzy** lub za grosze:

- **Subskrypcja** Claude (miesięczny abonament) — najprostszy start do nauki: płacisz stałą kwotę, bez liczenia tokenów przy każdej prośbie.
- **Modele darmowe** przez pośredników (np. warianty oznaczone „free” w OpenRouter) — wolniejsze, ale bez opłat.
- **Modele lokalne** (np. WebLLM w przeglądarce) — działają na Twoim komputerze, całkowicie za darmo, bez żadnego klucza.

OK - PUNKT KONTROLNY

Do przejścia Zeszytów 3–4 wystarczy **jedno** z powyższych. Zaczniemy od ścieżki najtańszej i najprostszej — nie potrzebujesz firmowej karty ani budżetu.

9. Najczęstsze obawy (FAQ)

Czy muszę umieć programować? Nie, żeby przejść ten pakiet. Im więcej rozumiesz, tym lepiej prowadzisz agenta — ale start nie wymaga pisania kodu.

Czy agent zepsuje mi komputer? Pracujesz w wydzielonym folderze projektu, a każdy krok w git da się cofnąć. Agent pyta o zgodę przed działaniami, które mogą coś nadpisać.

Czy to nie oszustwo, że „nie ja to napisałem”? Nie bardziej niż to, że architekt nie miesza betonu. Wartość jest w pomyśle, decyzjach i weryfikacji — a te są Twoje.

Czy AI nie zmyśla? Zdarza się (halucynacja). Dlatego cały pakiet kładzie nacisk na **weryfikację**: testy, uruchomienie, sprawdzenie na własne oczy.

Co, jeśli utknę? Agent jest też Twoim nauczycielem — możesz zapytać „nie rozumiem tego błędu, wyjaśnij po ludzku i zaproponuj rozwiązanie”. W Zeszycie 3 jest osobny rozdział „gdy coś wygląda inaczej”.

10. Co dalej

Masz już mapę pojęć. W kolejnych zeszytach:

- **Zeszyt 2** — prawdziwa historia: jak w 6 dni (i 30 minut koncepcji) powstała działająca aplikacja, kto za co odpowiadał i jaką metodą.
- **Zeszyt 3** — przygotowanie środowiska krok po kroku (konto, instalacje, pierwsze uruchomienie Claude Code) na Windows.
- **Zeszyt 4** — Twoja pierwsza aplikacja: powtórzysz całą metodę w miniaturze, z gotowymi promptami.

OK - PUNKT KONTROLNY

Zanim przejdziesz dalej, sprawdź, czy umiesz własnymi słowami odpowiedzieć: **czym różni się agent od czatu?** oraz **do czego służy Cowork, a do czego Claude Code?**

Jeśli tak — jesteś gotów na resztę pakietu.